

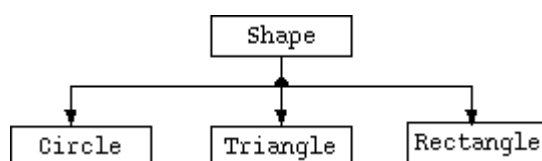
Överlagring av metoder

Vi skall nu se på en annan aspekt av ärvning, d.v.s. hur metoder kan överlagras. Med överlagring av metoder avses att en subclass kan ersätta en metod i en basklass med en egen metod, t.ex. för att specialisera metodens beteende till subclassen i fråga. Varför vill man göra något sådant, räcker det inte med att klasser kan anropa metoder ur sina basklasser och skapa nya metoder? Nej, i många fall vill man t.ex. i en basklass skapa ett enhetligt gränssnitt som definierar alla metoder som skall implementeras, och sedan är det upp till de olika subclasserna att implementera dessa metoder på ett sätt som lämpar sig för dem. Om vi tänker oss `Foo Factory`-exemplet ovan kunde vi tänka oss att definiera en metod `void doWork();` för klassen `Employee`. Men eftersom klassen `Employee` inte känner till sina subclasser och därför inte vet hur t.ex. en `Manager` arbetar kan man omöjligt i den metoden ge koden för hur `Manager` skall utföra sitt arbete. Däremot kan metoden innehålla kod för hur `Employee` skall utföra sitt arbete. Därefter kan klasserna `Manager` och `Secretary` *överlagra* metoden `doWork()` och där implementera den kod som representerar det arbete som dessa klasser utför.

Virtuella metoder

För att kunna överlagra metoder behövs i C++ ett koncept som heter *virtuella metoder*. En virtuell metod är en metod som kan överlagras. Inga andra metoder än virtuella metoder kan överlagras. Vi skall senare redogöra för varför det är så. En virtuell metod kännetecknas av att den har nyckelordet `virtual` före metoddefinitionen. För att titta närmare på virtuella metoder skall vi titta på en ny klasshierarki, nämligen en för att hantera olika geometriska figurer. De figurer vi vill representera är fyrkanter, trianglar och cirklar. Vi kommer att använda oss av klassen `Coordinate` som definierades i [Kapitel 14](#). Eftersom vi vet att varje av dessa figurer är, just det, en figur definieras en gemensam basklass `Shape` som de olika klasserna ärver. Denna klass innehåller några virtuella metoder som subclasserna förutsätts överlagra.

Figur 15-2. Klasshierarki över figurer



Först definieras klassen `Shape`:

```
class Shape {
public:
    // konstruktor
    Shape (const Coordinate & Origin);

    // virtuella metoder för area och omkrets
    virtual float area () const;
    virtual float circum () const;

    // position för figuren
    const Coordinate & origin () const;
    void setOrigin (const Coordinate & Origin);
};
```

```

private:
    // position
    Coordinate m-Origin;
};

Shape::Shape (const Coordinate & Origin) {
    m-Origin = Origin;
}

float Shape::area () const {
    return 0;
}

float Shape::circum () const {
    return 0;
}

const Coordinate & Shape::origin () const {
    return m-Origin;
}

void Shape::setOrigin (const Coordinate & Origin) {
    m-Origin = Origin;
}

```

Klassen `Shape` innehåller förutom de två virtuella metoderna `area()` och `circum()` (omkrets) även en konstruktör och en position. Positionen för figuren är en gemensam datamedlem, d.v.s. alla figurer har en viss position, därför är den definierad för klassen `Shape`. Subklasser kan accessera sin position via metoderna `origin()` och `setOrigin()`. Eftersom konstruktorn för `Shape` behöver en parameter måste subklasser anropa denna och ge en `Coordinate` som parameter. Subklasser skall överlagra de virtuella metoderna, därför är de implementerade så att de alltid returnerar 0 för dessa metoder. I framtiden kanske man även behöver olika andra datamedlemmar som är gemensamma för alla subklasser, t.ex. en färg `el.dyl.`, och då kan de placeras i klassen `Shape`. Vi kan om vi vill skapa objekt av typen `Shape`, men de är relativt värdelösa då vi inte kan göra något med dem. Istället definieras en klass `Circle`, som är exakt samma klass som i [Kapitel 14](#), men med de två virtuella metoderna definierade:

```

class Circle : public Shape {
public:
    // konstruktör
    Circle (const Coordinate & Origin, float Radius);

    // överlagrade metoder
    virtual float area () const;
    virtual float circum () const;

    // accessera radien
    float radius () const;
    void setRadius (float Radius);

private:
    // radie
    float m_Radius;
};

Circle::Circle (const Coordinate & Origin, float Radius) : Shape (Origin) {
    m_Radius = Radius;
}

float Circle::radius () const {
    return m_Radius;
}

```

```

}

void Circle::setRadius (float Radius) {
    m_Radius = Radius;
}

float Circle::area () const {
    // returnera en cirkles area
    return 3.14 * m_Radius * m_Radius;
}

float Circle::circum () const {
    // returnera en cirkles omkrets
    return 2 * 3.14 * m_Radius;
}

```

Metoderna måste i subklasser som ämnar överlagra dem även deklarerats och ges typen `virtual`. Man säger då åt kompilatorn att denna klass innehåller en implementation av de virtuella metoderna `area()` och `circum()`. Sedan då de implementeras är det inget speciellt jämfört med implementationen för `Shape`, inga extra definitioner. De skiljer sig dock till den grad att de returnerar en cirkels radie respektive omkrets. Vi definierar ännu en klass för att represententera en rektangel, nämligen `Rectangle`. Vi antar för enkelhetens skull att de rektanglar som används är rätvinkliga.

```

class Rectangle : public Shape {
public:
    // konstruktor
    Rectangle (const Coordinate & Origin,
               const Coordinate & Corner1, const Coordinate & Corner2,
               const Coordinate & Corner3, const Coordinate & Corner4);

    // överlagrade metoder
    virtual float area () const;
    virtual float circum () const;

private:
    // de fyra hörnpunkterna
    Coordinate m_Corners[4];
};

```

Vi implementerar inte dessa metoder, eftersom de är relativt lika de för klassen `Circle`.

Privata metoder

Vi får ett litet problem då vi skall beräkna omkretsen och radien för en `Rectangle`, nämligen hur skall vi få reda på avståndet mellan två punkter så att vi kan beräkna sidornas längd? Vi kan använda Pythagoras för det och göra beräkningen i metoderna `area()` och `circum()`, men det vore dumt att placera samma kod i två olika metoder. En lösning är att definiera en *privat metod* för klassen `Rectangle` som gör denna beräkning. Vi kan göra följande addition:

```

class Rectangle : public Shape {
    ...
private:
    // beräkna avstånd mellan två punkter
    float distance (const Coordinate & C1, const Coordinate & C2) const;

    // de fyra hörnpunkterna
    Coordinate m_Corners[4];
};

```

Metoden kan sedan implementeras på följande sätt:

```
float Rectangle::distance (const Coordinate & C1, const Coordinate & C2) const {
    float X = C1.x () - C2.x ();
    float Y = C1.y () - C2.y ();
    return sqrt ( (X * X) + (Y * Y));
}
```

Metoder kan alltså definieras som `private` om man vill. En sådan metod kan endast användas av klassen själv, ingen annan kommer åt den, inte ens subklasser. Privata metoder används vanligen för att flytta ofta använda operationer till skilda metoder, eller för att modularisera något internt förfarande. Vi kommer i framtiden att använda privata metoder nu och då i olika klasser. För att ovanstående kod skall vara kompilierbar bör man inkludera headefilen `math.h` och länka in biblioteket `libm` som innehåller matematikfunktionerna. Se [Kapitel 13](#) för information om hur man gör detta.

Om man tänker logiskt och objektorienterat hör ovanstående klass egentligen inte hemma i klassen `Rectangle`, utan i klassen `Coordinate`. Den kunde flyttas dit och definieras som en `public` metod. Då bör metoden ändras så att den tar endast en parameter och beräknar avståndet mellan den givna parameterkoordinaten och det aktuella objektet.

Dynamisk bindning

Vi kan för vår figur-klasshierarki tänka oss att skapa en funktion som skulle skriva ut arean och omkretsen för en given figur. Denna skulle vara en fristående funktion som inte tillhör någon klass. Vi får dock här ett litet problem. Vi har ju figurer av minst tre olika typer, nämligen `Circle`, `Rectangle` och `Triangle` (ej definierad). Måste vi då ha tre olika funktioner för att skriva ut dessa:

```
void printCircle (const Circle & C);
void printRectangle (const Rectangle & R);
void printTriangle (const Triangle & T);
```

Verkar en aning ineffektivt och arbetsdrygt. Istället kan vi ta fasta på att alla figurer har basklassen `Shape` och istället använda *dynamisk bindning* av metodanropen för `area()` och `circum()`. Vi definierar funktionen på följande sätt:

```
void print (const Shape & S) {
    cout << "Areal: " << S.area ()
         << ", omkretsen: " << S.circum () << endl;
}
```

Vi tar alltså som parameter en `Shape`. Vi kan då i anropande funktioner t.ex. göra följande:

```
int main () {
    Coordinate Origin ( 10, 10 );
    Coordinate Corner1 (0, 0 ), Corner2 ( 3, 0 );
    Coordinate Corner3 ( 3, 3 ), Corner4 ( 0, 3 );

    // skapa en cirkel och rektangel
    Circle C ( Origin, 5.0 );
    Rectangle R (Origin, Corner1, Corner2, Corner3, Corner4 );

    // skriv ut arean och omkretsen
    print ( C );
    print ( R );
}
```

Här ger vi parametrar av typen `Circle` och `Rectangle` till en funktion som skall ha parametrar av typen `Shape`. Verkar konstigt, eller hur. Kom då ihåg att varje subclass är även av sin basklass' typ, d.v.s. varje `Circle` är även en `Shape`. Tvärtom gäller förstås inte. Därför fungerar detta. När sedan metoderna `area()` och `circum()` exekveras i `print()` händer något som kallas *dynamisk bindning*. Programmet utför här en kontroll av vilken typ av klass parametern `s` egentligen är, och utför sedan denna metod. Tekniskt sätt så innehåller varje objekt en gömd tabell med adresser till de virtuella funktioner just det objektet skall använda. När en virtuell metod sedan utförs görs en kontroll mot denna tabell och adressen för den korrekta metoden hämtas och metoden utförs. Om en subclass inte har en överlagrad metod innehåller tabellen adressen för basklassens metod. I normala fall då det är frågan om en metod som inte är virtuell görs ingen dylik operation, istället utförs den inkompilerade koden direkt. Det är alltså till viss grad långsammare att ha virtuella metoder p.g.a. denna tabellsökning, men flexibiliteten hos virtuella metoder är enorm. Därför tvingas man i C++ även klart definiera vilka metoder som är virtuella så att kompilatorn vet vilka klasser som skall ha en metodtabell, och vilke metoder som skall placeras i tabellen.

Om en klass inte överlagrar en viss virtuell metod används den senast definierade versionen. Det är alltså inget tvång att överlagra en virtuell metod och den ursprungliga duger.

Dynamisk bindning och destruktorer

En klass som har en destruktör där man refererar till den genom dess basklass, måste ha destruktorn virtuell. Vi kan tänka oss ett exempel där vi har en funktion för att radera objekt som allokerats dynamiskt (se [Kapitel 17](#)). Den kan se ut på följande sätt:

```
void destroy (Shape * S) {
    delete S;
}
```

Vi kan anta att alla figur-klasser har en destruktör. I koden ovan kommer endast destruktorn för klassen `Shape` att utföras, inte destruktorn för den egentliga subclass som objektet representerar. Om vi skickar en `Circle` vill vi att destruktorn för `Circle` skall utföras först, och därefter destruktorn för `Shape`. Så händer dock inte här. Lösningen är att göra alla *subklassers* destruktorer virtuella. Då utförs de alla och i korrekt ordning. Att göra en destruktorn virtuell innebär endast att nyckelordet `virtual` placeras före destruktordefinitionen.

Abstrakta klasser

Vi har nu tittat på basklasser och olika subclasser och märkt att basklasser är en bra metod för att definiera ett *gränssnitt* som subclasser skall följa, och möjligen ändra funktionalitet för någon metod. Låt oss hypotetiskt tänka att det i en basklass, t.ex. `Shape` finns en eller flera metoder som det helt enkelt inte är möjligt att implementera, för att de är meningslösa på den nivån, eller om vi skulle vilja hindra programmeraren att instantiera objekt av basklassen. I vår enkla figur-hierarki är det fullt möjligt att skapa objekt av typen `Shape`, även om man inte har någon glädje av dem. Det finns ett sätt att hindra detta, samt att inte behöva implementera onödiga metoder. Man kan deklarera en metod att vara *rent virtuell*. En sådan metod innehåller ingen implementation överhuvudtaget. Om vi vill göra metoderna `area()` och `circum()` rent virtuella i `Shape` kan vi göra på följande sätt:

```
class Shape {
public:
    // konstruktor
    Shape (const Coordinate & Origin);

    // rent virtuella metoder för area och omkrets
    virtual float area () const = 0;
```

```
virtual float circum () const = 0;

// position för figuren
const Coordinate & origin () const;
void setOrigin (const Coordinate & Origin);

private:
    // position
    Coordinate m-Origin;
};
```

Enda skillnaden är att vi placerade `= 0` efter metoddefinitionen i klassdefinitionen. Metoderna är nu rent virtuella, och har därför ingen implementationskod i klassen `Shape`.

Det är inte tillåtet att instantiera en klass med rent virtuella metoder. Dylka klasser kallas *abstrakta basklasser*, eftersom de endast kan fungera som bas för andra klasser som ärer dem och överlagrar de metoder som är rent virtuella. På detta sätt kan vi hindra att programmerare instantierar vår klass `Shape`. Subklasser till en ABK måste inte överlagra alla rent virtuella metoder, men så länge som ens en metod är oimplementerad kan man inte instantiera en klass.

[Förutgående](#)

Konkret exempel

[Hem](#)[Upp](#)[Nästa](#)

Typer av ärvning